Validador sintáctico de URLs



Fundamentos Teóricos de Informática

Universidad de la Patagonia San Juan Bosco

Grupo 9

Gonzalez Bonetto Marcos
Cárcamo Mateo
Gomes dos Ramos Gonzalo

1. Introducción, objetivos y alcance **3**
1.1. Introducción 3
1.2. Objetivos 3
1.3. Alcance y limitaciones del proyecto 3
2. Fundamentación teórica **4**
2.1. Definición formal del AFND 4
2.2. Teoría de lenguajes y estructura gramatical 5
2.3. Expresión regular equivalente 6
2.4. Justificación de la elección de un AFND sobre un AFD 6
2.5. Tabla de Estados Autómata Finito No Determinista 6
3. Implementación 8
4. Pruebas realizadas 8
5. Bibliografía y herramientas utilizadas 9

1. Introducción, objetivos y alcance

1.1. Introducción

El análisis sintáctico y la validación de cadenas de caracteres constituyen uno de los pilares fundamentales de la teoría de lenguajes formales y la compilación. En el ámbito de las aplicaciones web y la comunicación en red, los **Localizadores Uniformes de Recursos (URL)** representan la estructura estándar para la identificación de recursos. La correcta verificación sintáctica de estas cadenas resulta crítica para prevenir errores de enrutamiento, vulnerabilidades de seguridad (como inyecciones o *phishing*) y garantizar la integridad de los datos de entrada.

El presente proyecto aborda la concepción, diseño e implementación de un sistema validador sintáctico de URLs fundado en la teoría de autómatas finitos. Mediante un **Autómata Finito No Determinista (AFND)**, el sistema evalúa si una cadena dada pertenece al lenguaje formal definido por las reglas sintácticas del estándar de URL. Adicionalmente, el proyecto integra una **Interfaz Gráfica de Usuario (GUI)** interactiva desarrollada en Python con Tkinter que visualiza de manera clara el diagrama de estados del autómata.

Link del repositorio: [Repositorio]

1.2. Objetivos

Objetivo general

Diseñar e implementar una herramienta computacional en Python basada en un **Autómata Finito No Determinista (AFND)** para la validación sintáctica de URLs, capaz de descomponer la estructura de la cadena evaluada y de presentar una representación visual responsiva del diagrama de estados para el análisis educativo y técnico.

1.3. Alcance y limitaciones del proyecto

Alcance soportado

- **Protocolo de red:** Reconocimiento opcional de los esquemas http:// y https://.
- **Estructura de host:** Validación de nombres de dominio principal y subdominios de niveles ilimitados delimitados por puntos (ej. www.mail.google.com), compuestos por caracteres alfanuméricos (a-z, A-Z, 0-9).
- **Extensiones dominiales:** Reconocimiento obligatorio de extensiones globales de nivel superior (**TLD**) de 3 letras (ej. .com, .org, .net) con código de país opcional (**ccTLD**) de 2 letras (ej. .com.ar), o directamente un ccTLD de 2 letras como única extensión (ej. .ar, .ai, .io).
- **Puerto TCP:** Identificación opcional de puertos especificados mediante : seguidos de un bloque de 1 a 5 dígitos numéricos (ej. :80, :8080).
- **Ruta de Recurso (*Path*):** Identificación opcional de la ruta de acceso especificada mediante / seguida de uno o más caracteres alfanuméricos (la barra sola, como en google.com/, no se acepta).

Limitaciones de la gramática actual (fuera del alcance)

- Parámetros de consulta (*query strings*) iniciados por ? (ej. ?id=123&lang=es).
- Extensiones de dominio de longitud distinta de 2 o 3 letras (ej. .info, .dev).
- Caracteres como guion (-) y guion bajo (_) en nombres de dominio y rutas.
- Validación del rango numérico del puerto (0–65535): el autómata solo verifica que sean de 1 a 5 dígitos, ya que un autómata finito no puede comparar valores numéricos.
- Direcciones IP puras (IPv4 / IPv6) en sustitución del nombre de dominio.

2. Fundamentación teórica

2.1. Definición formal del AFND

El núcleo del sistema de validación sintáctica de URLs se modela matemáticamente como un **Autómata Finito No Determinista (AFND)** definido mediante la quíntupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

1. Conjunto de Estados (Q):

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_{21}, q_7, subdominio, 1caracter_dominio, dominio, extension, 1letra, 2letra, 3letra, PAIS, 1LETRA, 2LETRA, puerto, 1num, 2num, 3num, 4num, 5num, ruta, q_{20}\}$$

2. Alfabetos de entrada (Σ):

- ***Letras (Σ_L):*** *{a, b,..., z, A, B,...,Z}*
- ***Dígitos (Σ_D):*** $\{0,1,2,3,4,5,6,7,8,9\}$
- ***Alfanumérico (Σ_A):*** $\Sigma_L \cup \Sigma_D$
- ***Símbolos (Σ_S):*** $\{:,/,.\}$
- ***Alfabeto Total:*** $\Sigma = \Sigma_A \cup \Sigma_S$

3. Estado inicial (q_0):

- q_0

4. Conjunto de Estados de Aceptación (F):

$$F = \{3letra, 2LETRA, 1num, 2num, 3num, 4num, 5num, q_{20}\}$$

2.2. Teoría de lenguajes y estructura gramatical

El lenguaje global reconocido por el autómata, $L(M) = L_{URL}$, es un **lenguaje regular** compuesto por la concatenación y opción de los siguientes lenguajes formales:

1. **Lenguaje del Protocolo (L_{prot}):**
   $L_{prot} = \{"http://","https://"\}$
2. **Lenguaje del Subdominio (L_{sub}):**

   $L_{sub} = \{w\ ."."\ /\ w \in\ \Sigma +_A\}$

3. **Lenguaje del Dominio Principal (L_{dom}):**

   $L_{dom} = \{w\ /\ w \in\ \Sigma +_A\}$

4. **Lenguaje del Primer TLD (L_{TLD1}):**

   $L_{TLD1} = \{"."\ .w\ /\ w \in\ \Sigma_L\ \wedge\ |w|\ =\ 3\ \}$

5. **Lenguaje del ccTLD / Código de País (L_{ccTLD}):**

   $L_{ccTLD} =\ \{"."\ .w\ /\ w \in\ \Sigma_L\ \wedge\ |w|\ =\ 2\ \}$

6. **Lenguaje del Puerto (L_{puerto}):**

   $L_{puerto} = \{":"\ .w\ /\ w \in\ \Sigma_D\ \wedge\ |w|\ \geq\ 1\ \wedge\ 5\ \geq\ |w|\ \}$

7. **Lenguaje de la Ruta / Path (L_{ruta}):**

   $L_{ruta} = \{"/"\ .w\ /\ w \in\ \Sigma +_A\ \}$

Expresión formal del lenguaje global:

$$L_{URL} = (L_{prot} \cup \{\lambda\}) \cdot (L_{sub}) *\cdot L_{dom} \cdot (L_{TLD1} \cdot (L_{ccTLD} \cup \{\lambda\}) \cup L_{ccTLD}) \cdot (L_{puerto} \cup \{\lambda\}) \cdot (L_{ruta} \cup \{\lambda\})$$

Nota: el bloque (L_TLD1 · (L_ccTLD ∪ {λ}) ∪ L_ccTLD) indica que el sufijo del host es un TLD de 3 letras con ccTLD opcional, o bien un ccTLD de 2 letras como única extensión (ej. claude.ai). L_sub se repite cero o más veces, lo que permite subdominios de varios niveles.

2.3. Expresión regular equivalente

(http://+https://+λ)(alfanumerico(alfanumerico)*.)*alfanumerico(alfanumerico)*(.letra letra letra(.letra letra+λ)+.letra letra)(:numero+:numero numero+:numero numero numero+:numero numero numero numero+:numero numero numero numero numero+λ)(/alfanumerico(alfanumerico)*+λ)

Gramática regular aceptada por el Autómata Finito

A partir del Autómata Finito diseñado para la validación de URLs se obtuvo la Gramática Regular equivalente, aplicando la conversión AF → GR (un no terminal por cada estado, como lo hace la herramienta **JFLAP**). La gramática es de **Tipo 3** según la Jerarquía de Chomsky (gramática regular derecha), cuyas producciones definen la estructura secuencial sintáctica del lenguaje aceptado.

1. Definición formal de la gramática G

La gramática se define formalmente como la 4-tupla $G = (V_N, V_T, P, S)$, donde:

- **Símbolo inicial:** $S = q0$
- **Conjunto de símbolos no terminales (V_N):**

  - $V_N = \{q0, q1, q2, q3, q4, q5, q6, q21, q7, subdominio, 1caracter_dominio, dominio, extension, 1letra, 2let$
- **Conjunto de Símbolos Terminales (V_T):**

  - $V_T = \{h, t, p, s, /, :, ., alfanumerico, letra, numero, \lambda\}$

2. Conjunto de Reglas de Producción (P)

A continuación se transcribe el conjunto completo de producciones, agrupadas para su correcta interpretación (el símbolo inicial es q0):

q0 → h q1 | alfanumérico subdominio | alfanumérico dominio

q1 → t q2

q2 → t q3

q3 → p q4

q4 → s q5 | : q6

q5 → : q6

q6 → / q21

q21 → / q7

q7 → alfanumerico subdominio | alfanumerico dominio

subdominio → alfanumerico subdominio | . 1caracter_dominio

1caracter_dominio → alfanumerico dominio

dominio → alfanumerico dominio | . extension | . 1caracter_dominio | . PAIS

extension → letra 1letra

1letra → letra 2letra

2letra → letra 3letra

3letra → . PAIS | : puerto | / ruta | λ

PAIS → letra 1LETRA

1LETRA → letra 2LETRA

2LETRA → : puerto | / ruta | λ

puerto → numero 1num

1num → numero 2num | / ruta | λ

2num → numero 3num | / ruta | λ

3num → numero 4num | / ruta | λ

4num → numero 5num | / ruta | λ

5num → / ruta | λ

ruta → alfanumerico q20

q20 → alfanumerico q20 | λ

2.4. Justificación de la Elección de un AFND sobre un AFD

1. **Ambigüedad inicial de prefijos:** Un carácter como 'h' puede ser el comienzo de http:// o el inicio del nombre de dominio (ej. host.com). En un AFND esto se resuelve mediante una transición hacia múltiples estados simultáneos desde q_0, evitando la duplicación masiva de estados que requeriría un AFD determinista.
2. **Ambigüedad del punto tras el dominio:** Después del dominio, un punto puede introducir otro nivel de subdominio (www.mail.google.com), el TLD de 3 letras (.com) o un ccTLD de 2 letras (.ai). El AFND explora las tres lecturas en paralelo desde el estado dominio (transiciones con '.' hacia extension, 1caracter_dominio y PAIS) y acepta si alguna rama llega a un estado final.
3. **Claridad del Grafo:** El modelo no determinista mapea conceptualmente los módulos sintácticos de la URL de forma directa e intuitiva

2.5. Tabla de Estados Autómata Finito No Determinista

Estado Actual (Q)	h	t	p	s	:	/	.	Letra	Digito
→q0	{q1,sub.,dom.}	{sub.,dom.}	{sub.,dom.}	{sub.,dom.}	∅	∅	∅	{sub.,dom.}	{sub.,dom.}
q1	∅	{q2}	∅	∅	∅	∅	∅	∅	∅
q2	∅	{q3}	∅	∅	∅	∅	∅	∅	∅
q3	∅	∅	{q4}	∅	∅	∅	∅	∅	∅
q4	∅	∅	∅	{q5}	{q6}	∅	∅	∅	∅
q5	∅	∅	∅	∅	{q6}	∅	∅	∅	∅
q6	∅	∅	∅	∅	∅	{q21}	∅	∅	∅
q21	∅	∅	∅	∅	∅	{q7}	∅	∅	∅
q7	{sub.,dom.}	{sub.,dom.}	{sub.,dom.}	{sub.,dom.}	∅	∅	∅	{sub.,dom.}	{sub.,dom.}
subdominio	{subdominio}	{subdominio}	{subdominio}	{subdominio}	∅	∅	{1caracter_dominio}	{subdominio}	{subdominio}
1caracter_dominio	{dominio}	{dominio}	{dominio}	{dominio}	∅	∅	∅	{dominio}	{dominio}
dominio	{dominio}	{dominio}	{dominio}	{dominio}	∅	∅	{extension, 1caracter_dominio, PAIS}	{dominio}	{dominio}
extension	{1letra}	{1letra}	{1letra}	{1letra}	∅	∅	∅	{1letra}	∅
1letra	{2letra}	{2letra}	{2letra}	{2letra}	∅	∅	∅	{2letra}	∅
2letra	{3letra}	{3letra}	{3letra}	{3letra}	∅	∅	∅	{3letra}	∅

*3letra	∅	∅	∅	∅	{puerto}	{ruta}	{PAIS}	∅	∅
PAIS	{1LETRA}	{1LETRA}	{1LETRA}	{1LETRA}	∅	∅	∅	{1LETRA}	∅
1LETRA	{2LETRA}	{2LETRA}	{2LETRA}	{2LETRA}	∅	∅	∅	{2LETRA}	∅
*2LETRA	∅	∅	∅	∅	{puerto}	{ruta}	∅	∅	∅
puerto	∅	∅	∅	∅	∅	∅	∅	∅	{1num}
*1num	∅	∅	∅	∅	∅	{ruta}	∅	∅	{2num}
*2num	∅	∅	∅	∅	∅	{ruta}	∅	∅	{3num}
*3num	∅	∅	∅	∅	∅	{ruta}	∅	∅	{4num}
*4num	∅	∅	∅	∅	∅	{ruta}	∅	∅	{5num}
*5num	∅	∅	∅	∅	∅	{ruta}	∅	∅	∅
ruta	∅	∅	∅	∅	∅	∅	∅	{q20}	{q20}
*q20	∅	∅	∅	∅	∅	∅	∅	{q20}	{q20}

3. Implementación

La implementación del proyecto se desarrolló en Python. Se empleó la librería **automata-lib** (automata.fa.nfa) para definir matemáticamente la tupla del AFND y validar formalmente las cadenas de entrada; la librería estándar **tkinter** para construir la interfaz gráfica mediante ventanas, contenedores y un lienzo (Canvas) deslizable; el módulo **Pillow** (PIL) para procesar, escalar y renderizar de manera responsiva el diagrama de estados dentro de la GUI; y la herramienta **graphviz** en un script independiente para dibujar y exportar el grafo de transiciones en formato PNG.

4. Pruebas realizadas

Palabra probada	Resultado esperado	Resultado obtenido	Motivo
google.com	Aceptada	Aceptada	Dominio con TLD válido
http://ejemplo.com.ar	Aceptada	Aceptada	Incluye protocolo y código de país.
https://ejemplo.com:8080	Aceptada	Aceptada	Incluye puerto.
https://ejemplo	Rechazada	Rechazada	Falta extensión de dominio.
https://com	Rechazada	Rechazada	Falta nombre de dominio.
https://ejemplo.com/ruta	Aceptada	Aceptada	Incluye ruta.
https://ejemplo.com/?id=10	Rechazada	Rechazada	Los parámetros de consulta están fuera del alcance actual.
www.mail.google.com	Aceptada	Aceptada	Subdominios de más de un nivel
https://console.cloud.google.com	Aceptada	Aceptada	Protocolo y dos niveles de subdominio
claude.ai	Aceptada	Aceptada	ccTLD de 2 letras como única extensión
miweb.ar	Aceptada	Aceptada	ccTLD de 2 letras como única extensión
www..google.com	Rechazada	Rechazada	Punto doble: falta un carácter entre los puntos.
google.c	Rechazada	Rechazada	Extensión incompleta (1 letra)
google.com:123456	Rechazada	Rechazada	Puerto de más de 5 dígitos
https://ejemplo.com/	Rechazada	Rechazada	La ruta exige al menos un carácter después de la barra.

5. Bibliografía y herramientas utilizadas

Librería para python “Autómata-lib”.

ChatGPT (OpenAI): utilizado como herramienta de apoyo para consultas conceptuales, revisión de documentación, organización del informe y asistencia durante el desarrollo.

Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3.ª ed.). Pearson.

Capítulos consultados: **2, “Finite Automata”**, y **3, “Regular Expressions and Languages”**.

Utilizado como referencia para la definición de autómatas finitos, AFND y su relación con los lenguajes y expresiones regulares.

(https://drive.google.com/drive/folders/1JpEYcm3lUfCJvKHao59IogYpPN2_Wj0K?usp=sharing)